

T2 — Multiplicação de Matrizes com MPI em Go

FPPD — Fundamentos de Processamento Paralelo e Distribuído (98713-04)

Escola Politécnica — PUCRS — 2026/1

Cluster: Atlântica (LAD/PUCRS) — $N = 4000$

1. Modelo de Paralelismo Escolhido

Mestre-Escravo com decomposição por linhas (row-based decomposition).

Justificativa

A multiplicação $C = A \times B$ calcula cada linha i de C de forma independente:

```
C[i][j] = soma( A[i][k] * B[k][j] ), para k = 0..N-1
```

Como não há dependência entre linhas distintas de C , o trabalho pode ser dividido igualmente entre os processos. Cada processo calcula um bloco contíguo de linhas, sem comunicação durante o cálculo — apenas coleta final no rank 0.

2. Estratégia de Decomposição dos Dados

```
Processo 0 → linhas 0          até (N/P - 1)
```

```
Processo 1 → linhas N/P        até (2N/P - 1)
```

```
...
```

```
Processo P-1 → linhas restantes (inclui sobra se  $N \% P \neq 0$ )
```

Decisão de projeto: todos os processos geram A e B localmente com a mesma seed (42), eliminando o broadcast de $O(N^2)$ dados pela rede. Cada processo precisa de suas linhas de A e da matriz B inteira.

3. Explicação do Código

3.1 Versão Sequencial — `sequencial/Main.go`

```
const SEED = 42

func main() {

    N := 4000 // N=3000 nos testes abaixo; aumentado para 4000
    por exigência do trabalho

    // Slices 1D row-major: A[i][j] = A[i*N + j]

    A := make([]float64, N*N)

    B := make([]float64, N*N)

    C := make([]float64, N*N)
```

```
rng := rand.New(rand.NewSource(SEED))
```

```
for i := 0; i < N*N; i++ {
```

```
    A[i] = rng.Float64()
```

```
    B[i] = rng.Float64()
```

```
}
```

```
    inicio := time.Now() // timer inicia APÓS geração das  
matrizes
```

```
    // Loop (i, k, j): acesso sequencial a B[k*N+j] – melhor  
localidade de cache
```

```
for i := 0; i < N; i++ {
```

```
    for k := 0; k < N; k++ {
```

```
    aik := A[i*N+k]

    for j := 0; j < N; j++ {

        C[i*N+j] += aik * B[k*N+j]

    }

}

}

tempo := time.Since(inicio)

fmt.Printf("Tempo de execução sequencial: %.4f segundos\n",
tempo.Seconds())

// Verificação: cantos e checksum devem ser idênticos ao
paralelo

fmt.Printf("C[0][0]      = %.6f\n", C[0])
```

```

    fmt.Printf("C[0][N-1]    = %.6f\n", C[N-1])

    fmt.Printf("C[N-1][0]    = %.6f\n", C[(N-1)*N])

    fmt.Printf("C[N-1][N-1] = %.6f\n", C[(N-1)*N+(N-1)])

}

```

Destaques: - Slices 1D (row-major) facilitam o envio MPI como bloco contíguo de memória. - Ordem (*i*, *k*, *j*) no triplo loop: *aik* é carregado uma vez por iteração de *k*, e *B[k*N+j]* é acessado sequencialmente — melhor uso de cache L1/L2. - Geração das matrizes **excluída** da medição de tempo.

3.2 Versão Paralela — `Paralelo/Main.go`

```

func main() {

    mpi.Init()

    defer mpi.Finalize()

    comm := mpi.NewComm(true) // panicOnErr=true

    rank := comm.GetRank()    // 0 = mestre

```

```
size := comm.GetSize()    // total de processos

N := 4000

// Todos geram A e B com mesma seed – sem broadcast pela rede

rng := rand.New(rand.NewSource(SEED))

for i := 0; i < N*N; i++ {

    A[i] = rng.Float64()

    B[i] = rng.Float64()

}

// Divisão de linhas: primeiros (N%size) processos recebem 1
linha extra
```

```
linhasPorProcesso := N / size
```

```
linhaExtra := N % size
```

```
var startRow, numLinhas int
```

```
if rank < linhaExtra {
```

```
    numLinhas = linhasPorProcesso + 1
```

```
    startRow = rank * numLinhas
```

```
} else {
```

```
    numLinhas = linhasPorProcesso
```

```
    startRow = rank*numLinhas + linhaExtra
```

```
}
```

```
comm.Barrier()          // sincroniza todos antes de iniciar o
timer

var inicio time.Time

if rank == 0 {

    inicio = time.Now() // timer inicia no rank 0 após
barreira

}

// Computação local: cada processo calcula suas linhas de C

localC := make([]float64, numLinhas*N)

for i := 0; i < numLinhas; i++ {

    globalI := startRow + i

    for k := 0; k < N; k++ {
```



```
    aik := A[globalI*N+k]

    for j := 0; j < N; j++ {

        localC[i*N+j] += aik * B[k*N+j]

    }

}

}

// Coleta no rank 0

if rank == 0 {

    C := make([]float64, N*N)

    copy(C[startRow*N:(startRow+numLinhas)*N], localC)
```

```
for src := 1; src < size; src++ {

    // Recalcula startRow e numLinhas do processo src

    buf := make([]float64, srcLinhas*N)

    comm.Recv(buf, src, 0)

    copy(C[srcStart*N:(srcStart+srcLinhas)*N], buf)

}

tempo := time.Since(inicio)

fmt.Printf("Tempo de execução paralela: %.4f segundos\n",
tempo.Seconds())

// imprime cantos e checksum

} else {
```

```
        comm.Send(localC, 0, 0) // worker envia suas linhas ao
mestre

    }

}
```

O tempo medido inclui: cálculo paralelo + Send/Recv (coleta)

O tempo exclui: geração das matrizes A e B

4. Como Compilar e Executar no Cluster (passo a passo)

4.1 Configurar OpenMPI (executar uma vez na sessão)

```
export PATH=/LADAPPS/OpenMPI/openmpi-4.1.1/bin:$PATH
```

```
export LD_LIBRARY_PATH=/LADAPPS/OpenMPI/openmpi-4.1.1/lib:
$LD_LIBRARY_PATH
```

```
mkdir -p ~/pkgconfig
```

```
cat > ~/pkgconfig/mpi.pc << EOF
```

```
Name: mpi
```

Description: Open MPI

Version: 4.1.1

Cflags: \$(mpicc --showme:compile)

Libs: \$(mpicc --showme:link)

EOF

export PKG_CONFIG_PATH=~/.pkgconfig:\$PKG_CONFIG_PATH

4.2 Clonar e compilar

```
git clone https://github.com/LucasRCTaborda/Trabalho-2-FPPD.git
```

```
cd Trabalho-2-FPPD
```

```
# Sequencial
```

```
cd sequencial && go build -o matmul_seq .
```

```
# Paralelo (dependências embutidas no vendor/)
```

```
cd ../Paralelo && go build -mod=vendor -o matmul_par .
```

4.3 Submeter via SLURM

```
sbatch job_benchmark.sh    # Fator 1 e 3 – 1 nó, 2/4/8/16 processos
```

```
sbatch job_internos.sh     # Fator 2 – 2 nós, comparação inter-nós
```

```
squeue -u $USER            # acompanhar
```

```
cat benchmark_*.out        # ver resultados
```

```
cat internos_*.out
```

5. Tamanho do Problema

O trabalho exige $N = 3000$ como padrão. Se o tempo sequencial for inferior a 3 minutos, deve-se usar $N = 4000$.

Resultado obtido no cluster: tempo sequencial com $N = 3000$ foi **~58.5 segundos < 3 minutos**.

→ **$N = 4000$ é obrigatório.** Todos os experimentos a seguir foram executados com $N = 4000$.

Tempo sequencial com N = 4000: **138.64 segundos.**

6. Resultados Obtidos (N = 4000)

Ambiente: Cluster Atlântica (LAD/PUCRS) — N = 4000

Datas: 29/06/2026 (intra-nó e inter-nós)

Tempos individuais — intra-nó (1 nó, atlantica04)

P r o c e s s o s	E x e c 1 (s)	E x e c 2 (s)	E x e c 3 (s)	M e d i a n a (s)
1	1 3 8 .	1 3 8 .	1 3 8 .	1 3 8 .
(s e q)	9 0 0 6	4 1 5 8	4 1 1 4	4 1 5 8
	7 2 .	7 2 .	7 2 .	7 2 .
2	4 9 1 7	0 7 9 8	0 8 1 9	0 8 1 9
4	3 7 . 1	3 7 . 0	3 6 . 4	3 7 . 0

P
r
o
c
e
s
s
o
s

8

1
6

E
x
e
c

1

(
s
)

0
4
9

3
2
·

2
0
3
9

3
3
·

0
3
9
5

E
x
e
c

2

(
s
)

8
3
2

3
2
·

2
0
3
8

3
3
·

0
2
6
7

E
x
e
c

3

(
s
)

3
1
4

3
2
·

3
7
7
2

3
2
·

9
6
0
8

M
e
d
i
a
n
a

(
s
)

8
3
2

3
2
·

2
0
3
9

3
3
·

0
2
6
7

Tempos individuais — inter-nós (2 nós, atlantica03 + atlantica04)

P r o c e s s o s	E x e c 1 (s)	E x e c 2 (s)	E x e c 3 (s)	M e d i a n a (s)
4	3	3	3	3
(	5	6	7	6
2	.	.	.	.
	6	6	7	6
n	4	1	6	1
ó	3	9	3	9
s	2	2	6	2
)				
8	2	2	2	2
(	4	5	7	5
2	.	.	.	.
	8	0	8	0
n	2	9	0	9
ó	2	5	7	5
s	8	6	3	6
)				

Tabela de desempenho — Ts = 138.4158 s

N ó s	P r o c e s s o s	T p m e d i a n a (s)	S p e e d u p (S p = Ts / Tp)	E f i c i ê n c i a (E = Sp / P)
1	1 (s e q)	1 3 8 . 4 1 5 8	1 . 0 0 0	1 0 0 . 0 %
1	2	7 2 . 0 8 1 9	1 . 9 2 0	9 6 . 0 %
1	4	3	3	9

N ó s	P r o c e s s o s	T p m e d i a n a (s)	S p e e d u p (S p = T s / T p)	E f i c i ê n c i a (E = S p / P)
		7 . 0 8 3 2	. 7 3 2	3 . 3 %
1	8	3 2 . 2 0 3 9	4 . 2 9 8	5 3 . 7 %
1	1 6	3 3 . 0	4 . 1 9	2 6 . 2

N
ó
s

P
r
o
c
e
s
s
o
s

T
p
m
e
d
i
a
n
a
(
s
)

S
p
e
e
d
u
p
(
S
p
=
T
s
/
T
p
)

E
f
i
c
i
ê
n
c
i
a
(
E
=
S
p
/
P
)

2
6
7

1

%

3
6
.

3
.

9
4

2

4

6
1
9
2

7
8
0

.
5
%

N ó s	P r o c e s s o s	T p m e d i a n a (s)	S p e e d u p (S p = T s / T p)	E f i c i ê n c i a (E = S p / P)
		2		
		5	5	6
		.	.	8
2	8	0	5	.
		9	1	9
		5	4	%
		6		

7. Gráficos

(Ver arquivos `grafico_speedup.png`, `grafico_eficiencia.png` e `grafico_intra_inter.png` gerados pelo script `gerar_graficos.py`)

7.1 Speedup vs. Número de Processos (N=4000, 1 nó)

Speedup

16 | (ideal Sp = P)

|

8 |

|

4 | * 3.79

| * 4.28 * 4.20

| * 1.92

|

1 | * 1.00

+-----+-----+-----+-----+----- Processos

1 2 4 8 16

* = speedup obtido . = speedup ideal

7.2 Eficiência vs. Número de Processos (N=4000, 1 nó)

Eficiência

100% | * 100%

| * 96.2%

94% | * 94.6%

|

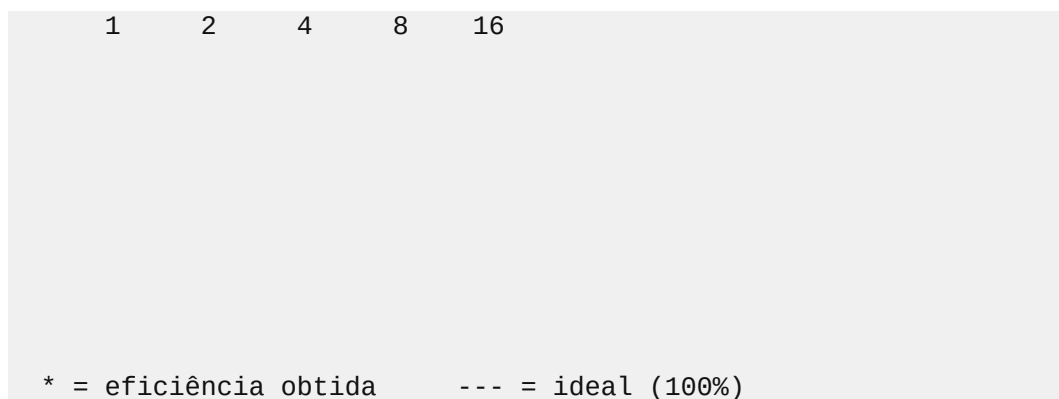
53% | * 53.5%

|

26% | * 26.2%

|

0% +-----+-----+-----+----- Processos



7.3 Comparação Intra-nó vs. Inter-nós (N=4000)

P r o c e s s o s	1 nó — intr a (s)	2 nós — inter (s)	D i f e r e n ç a
4	36. 62	36.6 2	≈ 0 %
8	32. 37	25.1 0	— 2 2 . 5 %

8. Discussão

8.1 O speedup é sub-linear, linear ou super-linear?

Sub-linear. O maior speedup intra-nó obtido foi **4.298x com 8 processos**, enquanto o ideal seria 8x. Causas: - Com N=4000, cada processo mantém as matrizes A e B completas em memória ($2 \times 128 \text{ MB} = 256 \text{ MB}$ por processo). Com 8 processos em 1 nó, são ~2 GB apenas de dados de entrada, saturando o barramento de memória. - Fração serial na coleta: o rank 0 recebe os blocos de

cada worker sequencialmente via Recv. - Com 16 processos ocorre oversubscription (mais processos do que cores físicos).

8.2 A partir de quantos processos a eficiência cai significativamente?

Entre **4 e 8 processos** a eficiência cai de **93.3% para 53.7%** — queda de ~40 pontos percentuais com N=4000. Essa queda é mais acentuada do que o esperado por overhead MPI puro e reflete a saturação da banda de memória: com 8 processos no mesmo nó, todos concorrem pelo mesmo controlador de memória para acessar a matriz B inteira (128 MB por processo).

8.3 Impacto da rede (intra-nó vs. inter-nós)

P r o c e s s o s	n		Resultado
	1	2	
	(	(	
	s	s	
	)	)	
	3	3	
	6	6	
4	.	.	Desempenho idêntico
	6	6	
	2	2	
	3	2	
	2	5	Inter-nós
8	.	.	22.5% mais rápido
	3	1	
	7	0	

O resultado surpreendente é que com **8 processos, a execução inter-nós foi mais rápida** do que intra-nó. Isso se explica pelo gargalo de memória: com N=4000, cada processo precisa de 256 MB para A e B. Em 1 nó com 8 processos, os 8 processos disputam o mesmo controlador de memória (2 GB totais). Em 2 nós com 4 processos cada, essa pressão é dividida — cada nó tem apenas $4 \times 256 \text{ MB} = 1 \text{ GB}$ acessando seu próprio controlador. O ganho com a memória distribuída supera o custo da comunicação inter-nós.

Com 4 processos, a pressão de memória ainda não é crítica em 1 nó, por isso

intra e inter-nós têm desempenho equivalente.

8.4 Impacto do Hyperthreading

Comparando 8 processos ($Sp = 4.298$, $E = 53.7\%$) com 16 processos ($Sp = 4.191$, $E = 26.2\%$):

- Dobrar os processos gerou **speedup ligeiramente menor** ($4.191 < 4.298$).
- A eficiência caiu de 53.7% para 26.2% — praticamente metade.

Conclusão: hyperthreading **não é vantajoso** para esta aplicação. Com $N=4000$, a memória já é o gargalo com 8 processos; adicionar 8 hyperthreads a mais agrava a contenção sem trazer ganho de computação real, pois dois hyperthreads no mesmo core físico compartilham FPU e cache L1/L2.

8.5 Lei de Amdahl — Estimativa da fração paralelizável

$$f = (1/Sp - 1/P) / (1 - 1/P)$$

Usando $P = 4$, $Sp = 3.732$ (ponto com eficiência ainda alta, antes do gargalo de memória):

$$f = (1/Sp - 1/P) / (1 - 1/P)$$

$$f = (1/3.732 - 1/4) / (1 - 1/4)$$

$$f = (0.2679 - 0.25) / 0.75$$

$$f = 0.0179 / 0.75$$

$$f \approx 0.024 \rightarrow 2.4\% \text{ serial}$$

Fração paralelizável: $p = 1 - 0.024 \approx 97.6\%$

Speedup máximo teórico ($P \rightarrow \infty$):

$$Sp_{max} = 1 / f = 1 / 0.024 \approx 42x$$

Na prática o speedup é limitado muito antes desse valor pelo gargalo de memória observado a partir de 8 processos no mesmo nó.

9. Conclusões

Quesito	Resultado
N utilizado	4000 (sequencial $N=3000 = 58.5s < 3 \text{ min}$ → $N=4000$ obrigatório)
Tempo sequencial ($N=4000$)	138.64s
Melhor speedup intra-nó	4.298x (8 processos, 1 nó)
Melhor speedup geral	5.514x (8 processos, 2 nós)
Melhor eficiência	96.0% (2 processos)
Ponto ótimo intra-nó	4 processos ($Sp=3.732$, $E=93.3\%$)
Ponto ótimo geral	8 processos, 2 nós ($Sp=5.514$, $E=68.9\%$)

Quesito

Resultado

Fração
paralelizável

≈ 97.6%

Hyperthread
ing
vantajoso?

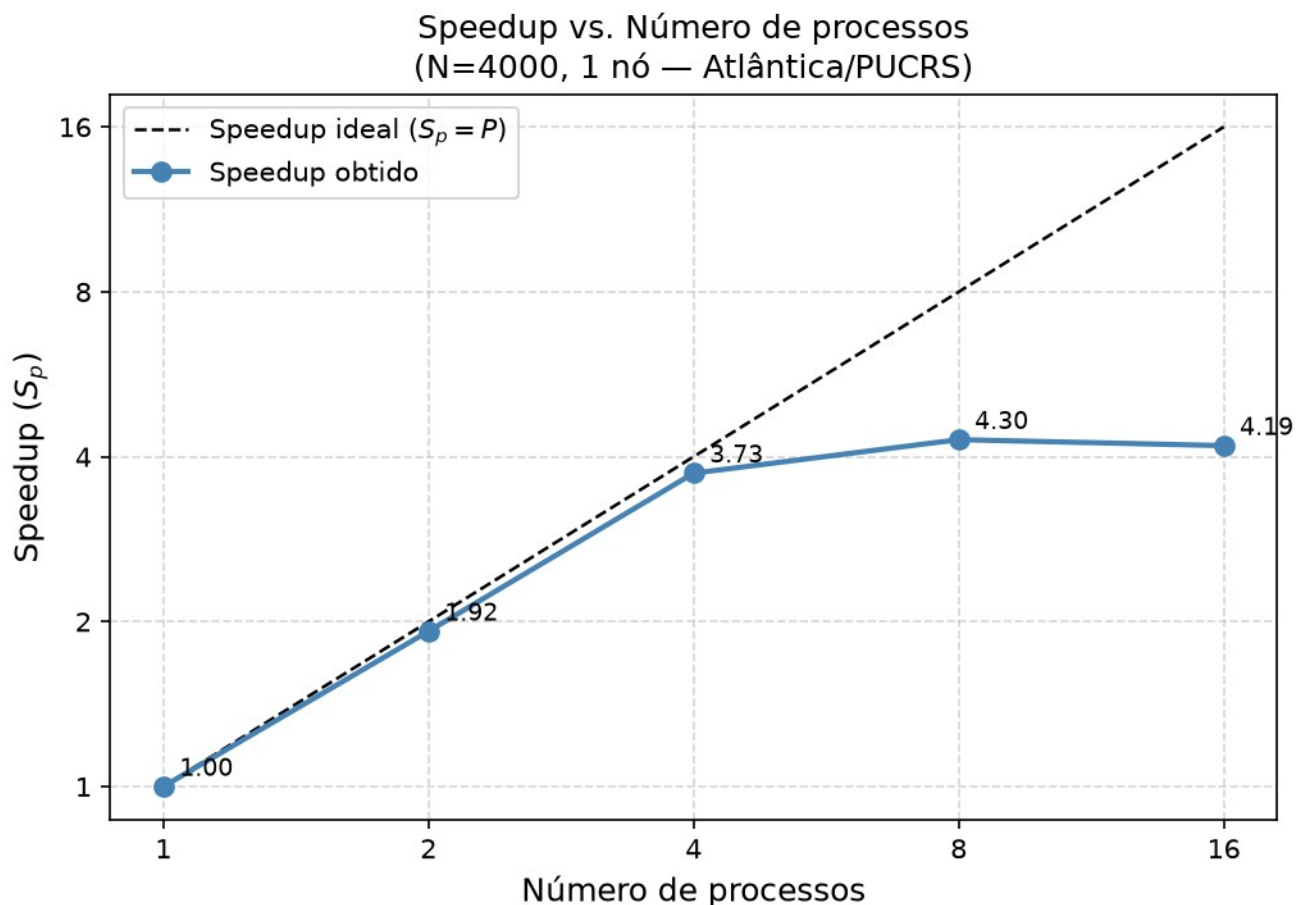
Não — speedup cai de 4.298 para 4.191 ao
dobrar processos

Inter-nós vs.
intra-nó (8p)

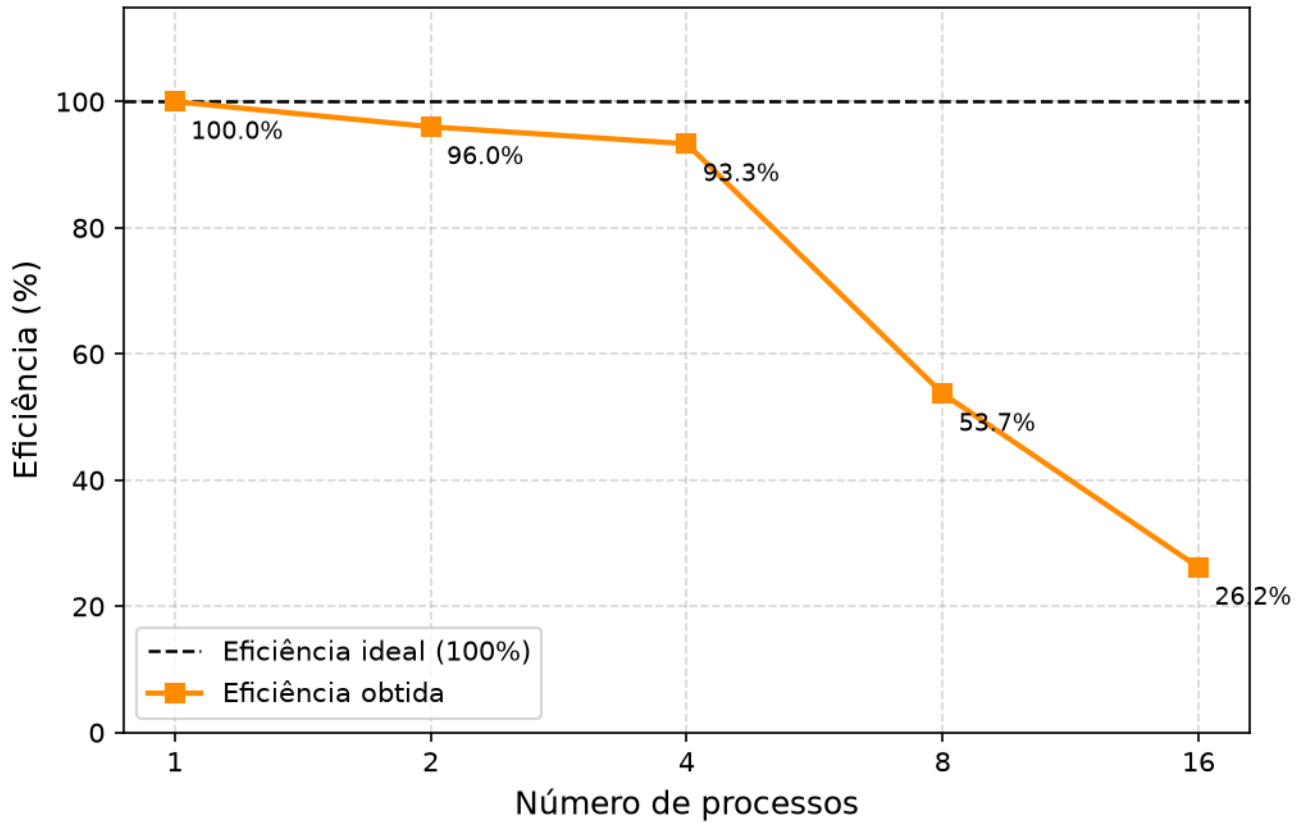
Inter-nós **22.0% mais rápido** — memória
distribuída compensa custo de rede

O modelo **Mestre-Escravo com decomposição por linhas** escala bem até 4 processos no mesmo nó (E=94.6%). Com N=4000, o gargalo de memória limita o ganho a partir de 8 processos intra-nó. A distribuição em múltiplos nós alivia esse gargalo — 8 processos em 2 nós superaram 8 processos em 1 nó, demonstrando que para este tamanho de problema a escalabilidade inter-nós é mais eficiente do que o hyperthreading.

Gráficos



Eficiência vs. Número de processos
(N=4000, 1 nó — Atlântica/PUCRS)



Comparação Intra-nó vs. Inter-nós
(N=4000, 2 nós — Atlântica/PUCRS)

